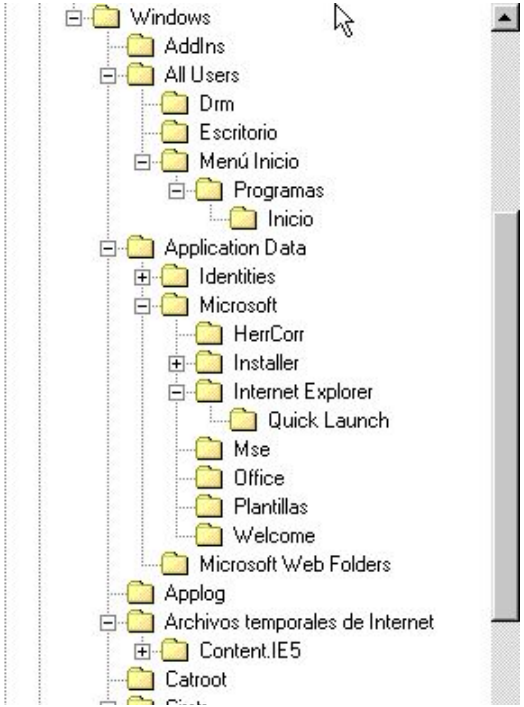
Árvore Binária de Busca

Estruturas de Dados

adalberto@computacao.ufcg.edu.br

Problema

E se o sistema de arquivos fosse armazenado em uma estrutura linear?



Solução

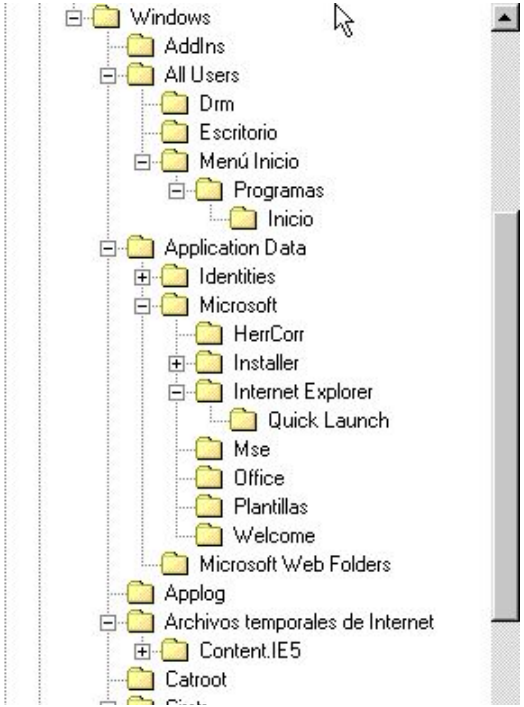
Hierarquizar a informação
Nós em hierarquia

Estrutura não-linear

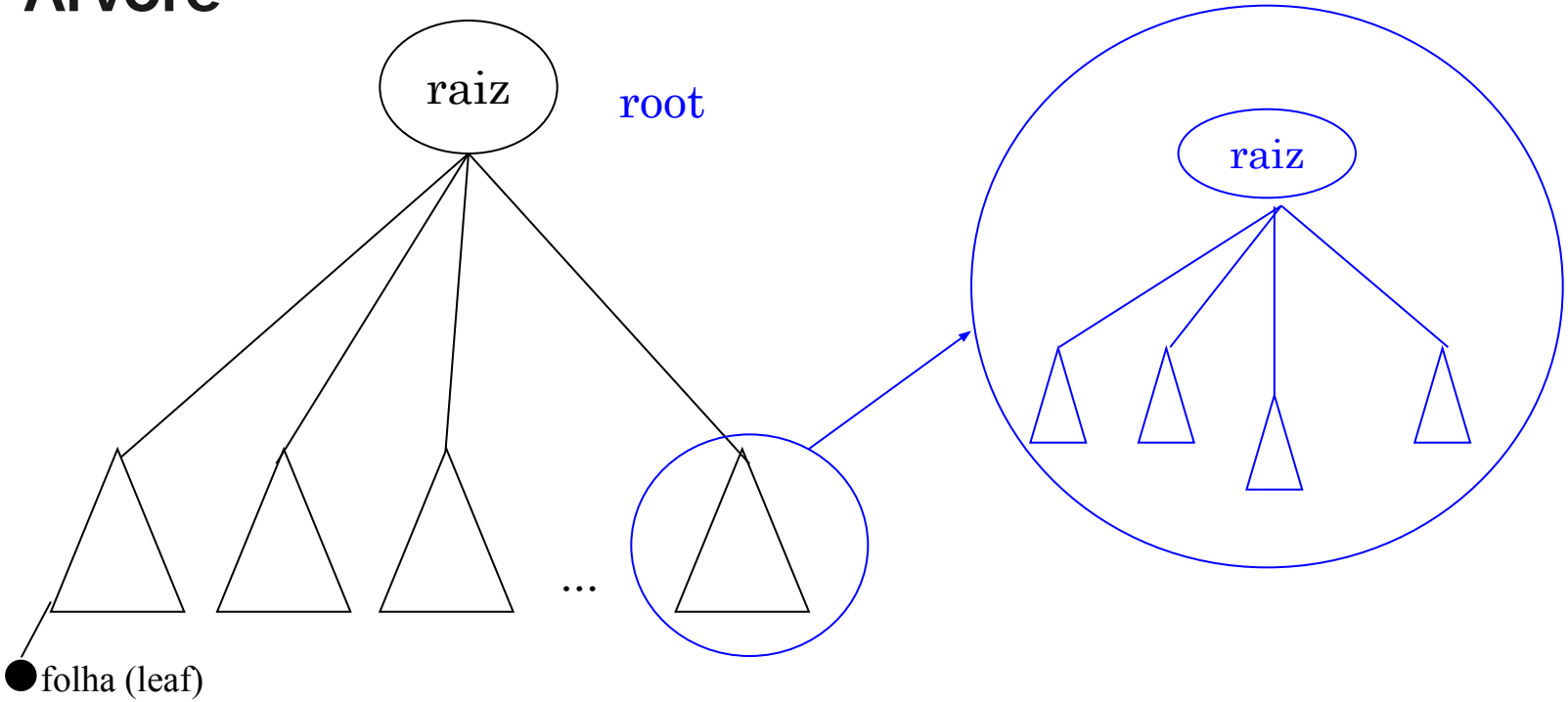
Não possui ciclos

#Arestas = #Nós - 1

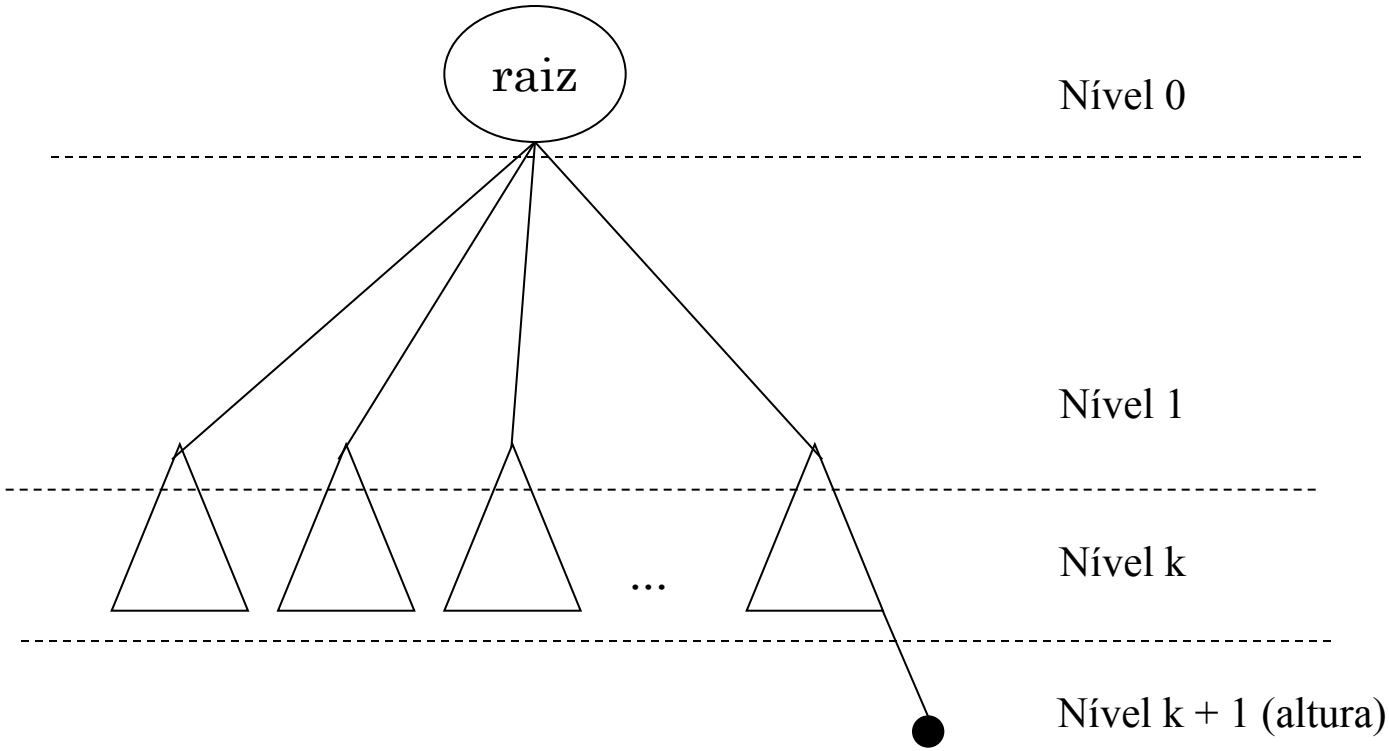
Pode ser vista por níveis



Árvore



Árvore



Árvore Binária

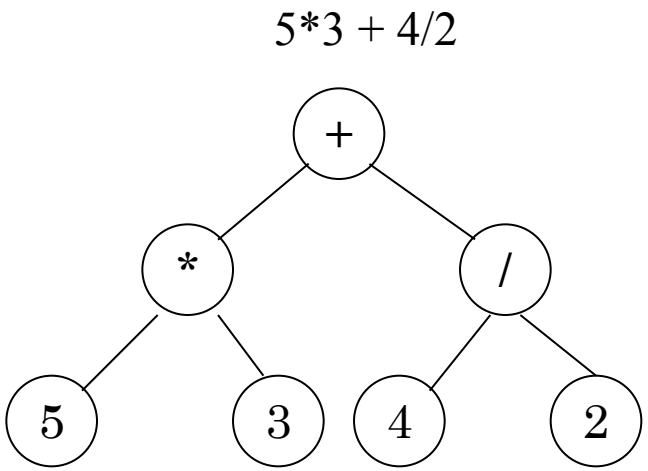
Ou não tem nó algum (**árvore vazia**)

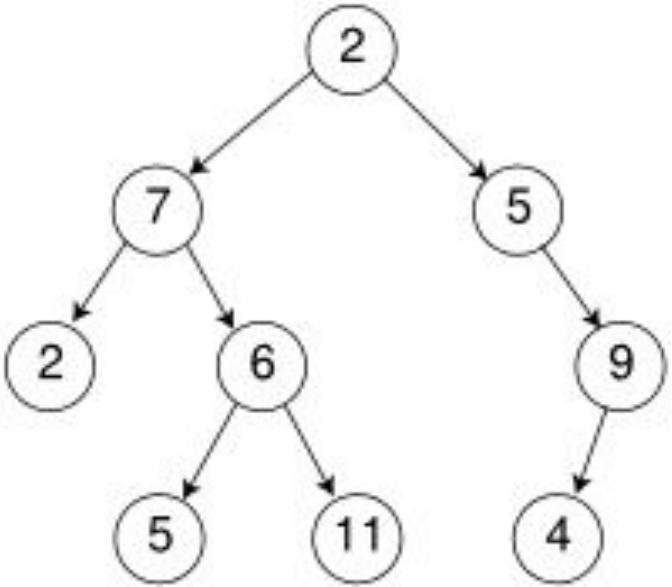
Ou possui um nó **raiz** com no máximo dois filhos: sub-árvore/filho a **esquerda** e/ou a **direita** (apontadores dos pais para os filhos)

Cada nó ó possui graus: 0, 1 ou 2 (define a aridade)

Só existe um caminho da raiz para qualquer nó da árvore

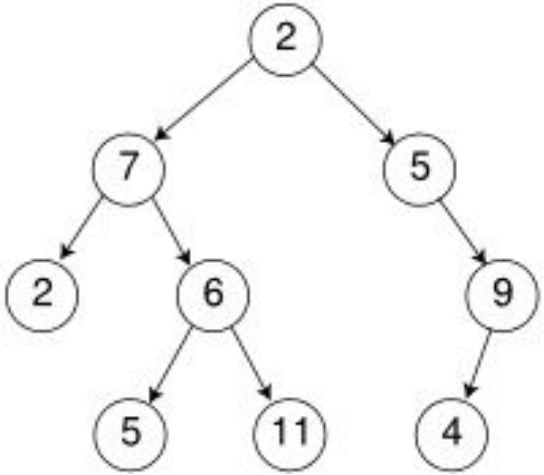
Árvore Binária (exemplos)





Árvore Estritamente Binária (AEB)

Cada nó possui grau 0 ou 2



Árvore Estritamente Binária (AEB)

Qual a relação entre o número de nós e número de folhas em uma AEB?

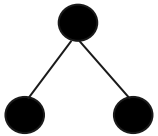
Árvore Estritamente Binária (AEB)

Qual a relação entre o número de nós e número de folhas em uma AEB?

#nós	#folhas
1	1

Árvore Estritamente Binária (AEB)

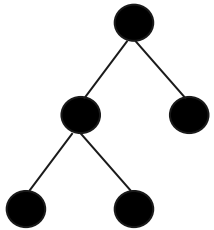
Qual a relação entre o número de nós e número de folhas em uma AEB?



#nós	#folhas
1	1
3	2

Árvore Estritamente Binária (AEB)

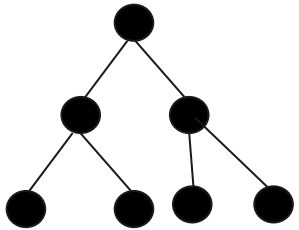
Qual a relação entre o número de nós e número de folhas em uma AEB?



#nós	#folhas
1	1
3	2
5	3

Árvore Estritamente Binária (AEB)

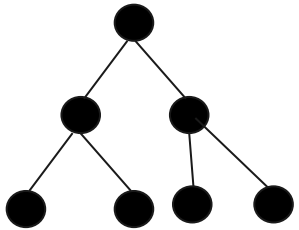
Qual a relação entre o número de nós e número de folhas em uma AEB?



#nós	#folhas
1	1
3	2
5	3
7	4

Árvore Estritamente Binária (AEB)

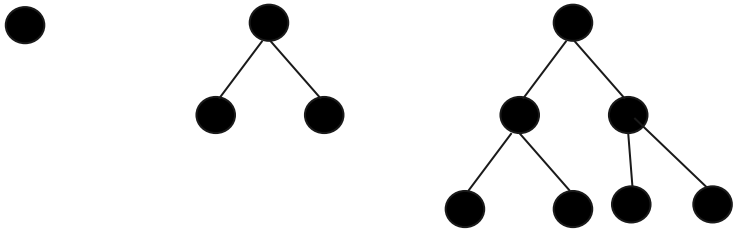
Qual a relação entre o número de nós e número de folhas em uma AEB?



#nós	#folhas
1	1
3	2
5	3
7	4
2f - 1	**f**

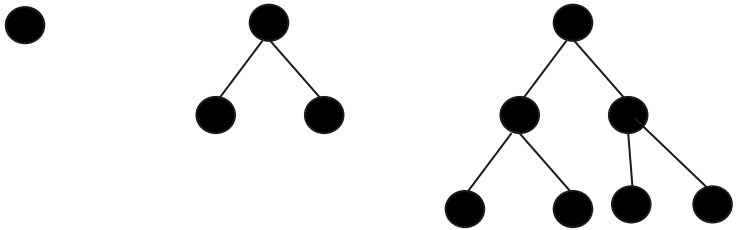
Árvore Binária Completa

O último nível está sempre completo!



Árvore Binária Completa

O último nível está sempre completo!

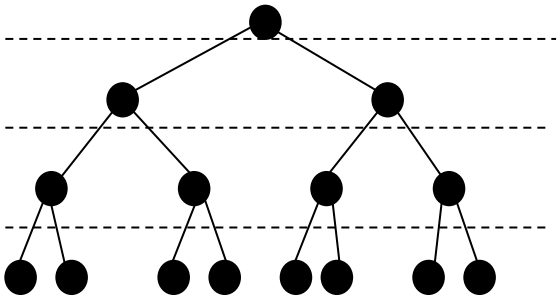


Qual a relação entre altura e número de nós em cada nível?

Qual a relação entre altura e número de nós total?

Árvore Binária Completa

O último nível está sempre completo!



nível 0 – nos:1 – total:1

nível 1 – nos:2 – total:3

nível 2 – nos:4 – total:7

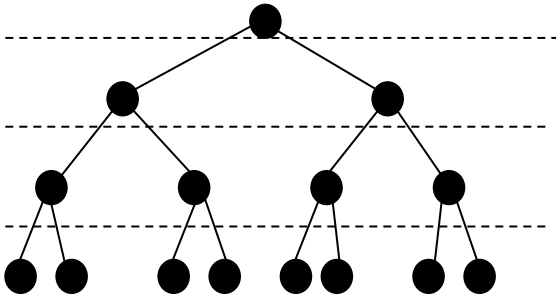
nível 3 – nos:8 – total:15

Qual a relação entre altura e número de nós em cada nível?

Qual a relação entre altura e número de nós total?

Árvore Binária Completa

O último nível está sempre completo!



nível 0 – nos:1 – total:1

nível 1 – nos:2 – total:3

nível 2 – nos:4 – total:7

nível 3 – nos:8 – total:15

h $\quad 2^h \quad 2^{h+1} - 1$

Qual a relação entre altura e número de nós em cada nível?

Qual a relação entre altura e número de nós total?

Formas de Percurso

Formas de Percursos

Pré-ordem (RAIZ, ESQ, DIR)

- **Visita** raiz
- Percorre sub-árvore esquerda em pré-ordem
- Percorre sub-árvore direita em pré-ordem

Em-ordem (simétrica) (ESQ, RAIZ, DIR)

- Percorre sub-árvore esquerda em ordem simétrica
- **Visita** raiz
- Percorre sub-árvore direita em ordem simétrica

Pós-ordem (ESQ, DIR, RAIZ)

- Percorre sub-árvore esquerda em pós-ordem
- Percorre sub-árvore direita em pós-ordem
- **Visita** raiz

Formas de Percursos

Pré-ordem (RAIZ, ESQ, DIR)

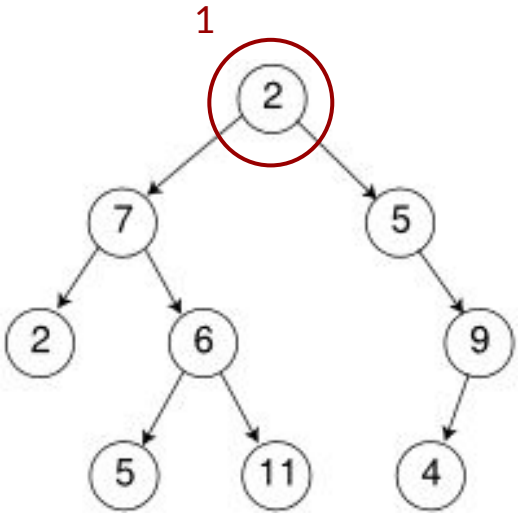
- **Visita** raiz
- Percorre sub-árvore esquerda em pré-ordem
- Percorre sub-árvore direita em pré-ordem

Em-ordem (simétrica) (ESQ, RAIZ, DIR)

- Percorre sub-árvore esquerda em ordem simétrica
- **Visita** raiz
- Percorre sub-árvore direita em ordem simétrica

Pós-ordem (ESQ, DIR, RAIZ)

- Percorre sub-árvore esquerda em pós-ordem
- Percorre sub-árvore direita em pós-ordem
- **Visita** raiz



2

Formas de Percursos

Pré-ordem (RAIZ, ESQ, DIR)

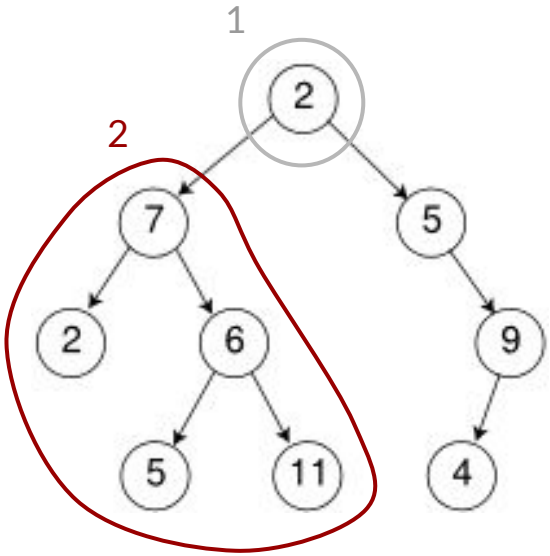
- **Visita** raiz
- Percorre sub-árvore esquerda em pré-ordem
- Percorre sub-árvore direita em pré-ordem

Em-ordem (simétrica) (ESQ, RAIZ, DIR)

- Percorre sub-árvore esquerda em ordem simétrica
- **Visita** raiz
- Percorre sub-árvore direita em ordem simétrica

Pós-ordem (ESQ, DIR, RAIZ)

- Percorre sub-árvore esquerda em pós-ordem
- Percorre sub-árvore direita em pós-ordem
- **Visita** raiz



2 7 ...

Formas de Percursos

Pré-ordem (RAIZ, ESQ, DIR)

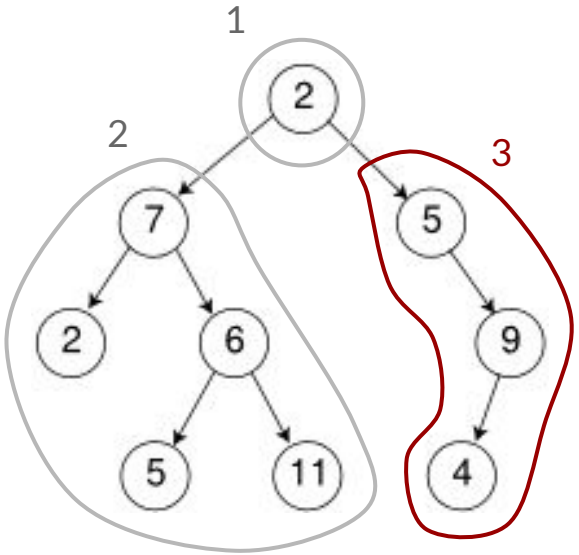
- **Visita** raiz
- Percorre sub-árvore esquerda em pré-ordem
- Percorre sub-árvore direita em pré-ordem

Em-ordem (simétrica) (ESQ, RAIZ, DIR)

- Percorre sub-árvore esquerda em ordem simétrica
- **Visita** raiz
- Percorre sub-árvore direita em ordem simétrica

Pós-ordem (ESQ, DIR, RAIZ)

- Percorre sub-árvore esquerda em pós-ordem
- Percorre sub-árvore direita em pós-ordem
- **Visita** raiz



2 **7 ...** **5 ...**

simulação

Formas de Percursos

Pré-ordem (RAIZ, ESQ, DIR)

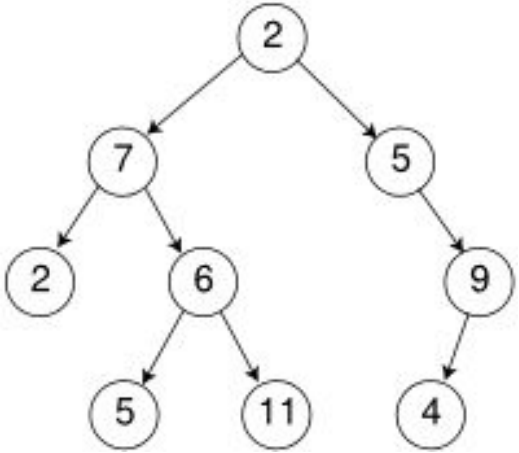
- **Visita** raiz
- Percorre sub-árvore esquerda em pré-ordem
- Percorre sub-árvore direita em pré-ordem

Em-ordem (simétrica) (ESQ, RAIZ, DIR)

- Percorre sub-árvore esquerda em ordem simétrica
- **Visita** raiz
- Percorre sub-árvore direita em ordem simétrica

Pós-ordem (ESQ, DIR, RAIZ)

- Percorre sub-árvore esquerda em pós-ordem
- Percorre sub-árvore direita em pós-ordem
- **Visita** raiz



2 7 2 6 5 11 5 9 4

Formas de Percursos

Pré-ordem (RAIZ, ESQ, DIR)

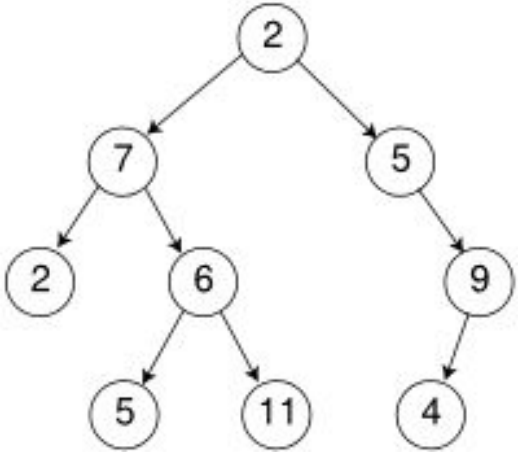
- **Visita** raiz
- Percorre sub-árvore esquerda em pré-ordem
- Percorre sub-árvore direita em pré-ordem

Em-ordem (simétrica) (ESQ, RAIZ, DIR)

- Percorre sub-árvore esquerda em ordem simétrica
- **Visita** raiz
- Percorre sub-árvore direita em ordem simétrica

Pós-ordem (ESQ, DIR, RAIZ)

- Percorre sub-árvore esquerda em pós-ordem
- Percorre sub-árvore direita em pós-ordem
- **Visita** raiz



Formas de Percursos

Pré-ordem (RAIZ, ESQ, DIR)

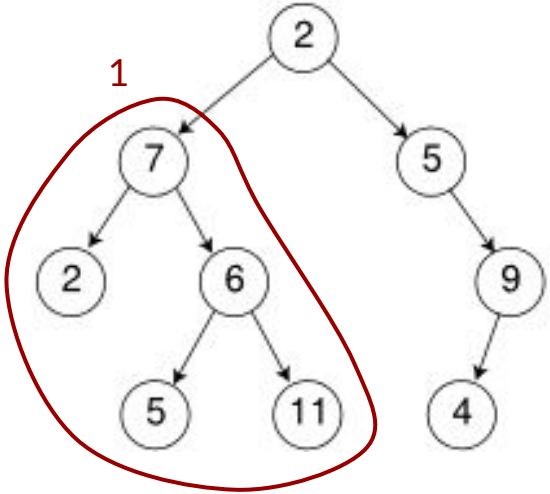
- **Visita** raiz
- Percorre sub-árvore esquerda em pré-ordem
- Percorre sub-árvore direita em pré-ordem

Em-ordem (simétrica) (ESQ, RAIZ, DIR)

- Percorre sub-árvore esquerda em ordem simétrica
- **Visita** raiz
- Percorre sub-árvore direita em ordem simétrica

Pós-ordem (ESQ, DIR, RAIZ)

- Percorre sub-árvore esquerda em pós-ordem
- Percorre sub-árvore direita em pós-ordem
- **Visita** raiz



Formas de Percursos

Pré-ordem (RAIZ, ESQ, DIR)

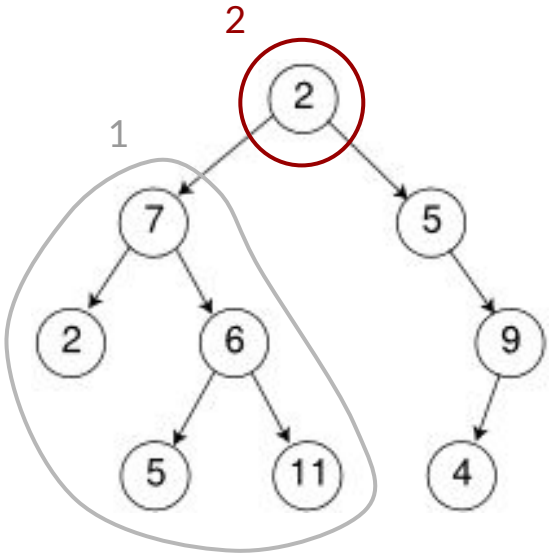
- **Visita** raiz
- Percorre sub-árvore esquerda em pré-ordem
- Percorre sub-árvore direita em pré-ordem

Em-ordem (simétrica) (ESQ, RAIZ, DIR)

- Percorre sub-árvore esquerda em ordem simétrica
- **Visita** raiz
- Percorre sub-árvore direita em ordem simétrica

Pós-ordem (ESQ, DIR, RAIZ)

- Percorre sub-árvore esquerda em pós-ordem
- Percorre sub-árvore direita em pós-ordem
- **Visita** raiz



Formas de Percursos

Pré-ordem (RAIZ, ESQ, DIR)

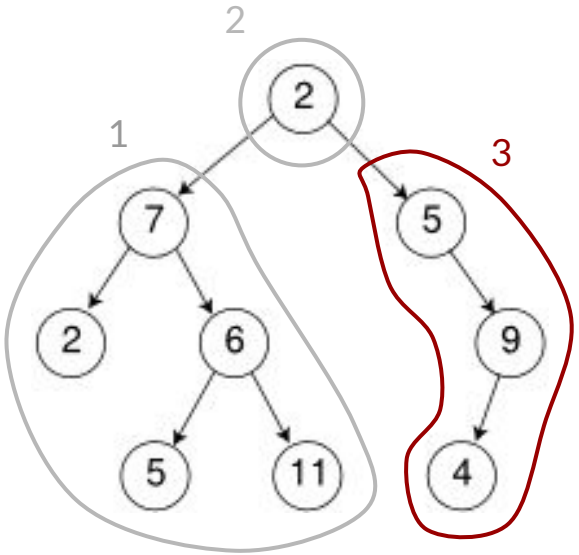
- **Visita** raiz
- Percorre sub-árvore esquerda em pré-ordem
- Percorre sub-árvore direita em pré-ordem

Em-ordem (simétrica) (ESQ, RAIZ, DIR)

- Percorre sub-árvore esquerda em ordem simétrica
- **Visita** raiz
- Percorre sub-árvore direita em ordem simétrica

Pós-ordem (ESQ, DIR, RAIZ)

- Percorre sub-árvore esquerda em pós-ordem
- Percorre sub-árvore direita em pós-ordem
- **Visita** raiz



simulação

Formas de Percursos

Pré-ordem (RAIZ, ESQ, DIR)

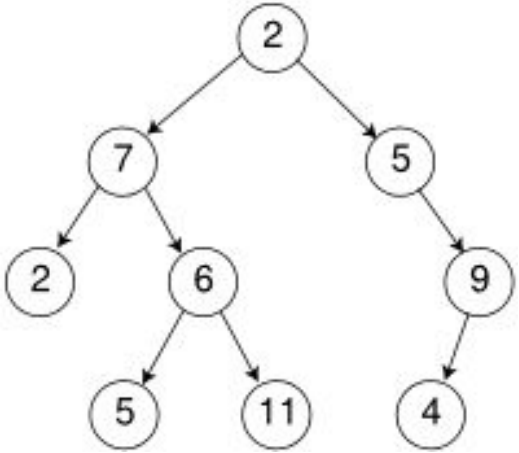
- **Visita** raiz
- Percorre sub-árvore esquerda em pré-ordem
- Percorre sub-árvore direita em pré-ordem

Em-ordem (simétrica) (ESQ, RAIZ, DIR)

- Percorre sub-árvore esquerda em ordem simétrica
- **Visita** raiz
- Percorre sub-árvore direita em ordem simétrica

Pós-ordem (ESQ, DIR, RAIZ)

- Percorre sub-árvore esquerda em pós-ordem
- Percorre sub-árvore direita em pós-ordem
- **Visita** raiz



2 7 5 6 11 2 5 4 9

simulação

Formas de Percursos

Pré-ordem (RAIZ, ESQ, DIR)

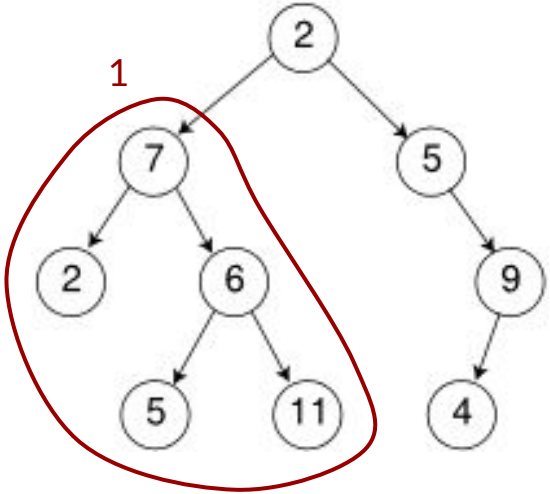
- **Visita** raiz
- Percorre sub-árvore esquerda em pré-ordem
- Percorre sub-árvore direita em pré-ordem

Em-ordem (simétrica) (ESQ, RAIZ, DIR)

- Percorre sub-árvore esquerda em ordem simétrica
- **Visita** raiz
- Percorre sub-árvore direita em ordem simétrica

Pós-ordem (ESQ, DIR, RAIZ)

- Percorre sub-árvore esquerda em pós-ordem
- Percorre sub-árvore direita em pós-ordem
- **Visita** raiz



Formas de Percursos

Pré-ordem (RAIZ, ESQ, DIR)

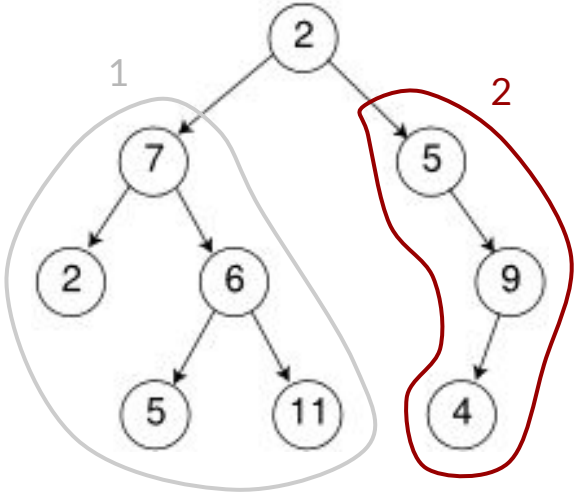
- **Visita** raiz
- Percorre sub-árvore esquerda em pré-ordem
- Percorre sub-árvore direita em pré-ordem

Em-ordem (simétrica) (ESQ, RAIZ, DIR)

- Percorre sub-árvore esquerda em ordem simétrica
- **Visita** raiz
- Percorre sub-árvore direita em ordem simétrica

Pós-ordem (ESQ, DIR, RAIZ)

- Percorre sub-árvore esquerda em pós-ordem
- Percorre sub-árvore direita em pós-ordem
- **Visita** raiz



Formas de Percursos

Pré-ordem (RAIZ, ESQ, DIR)

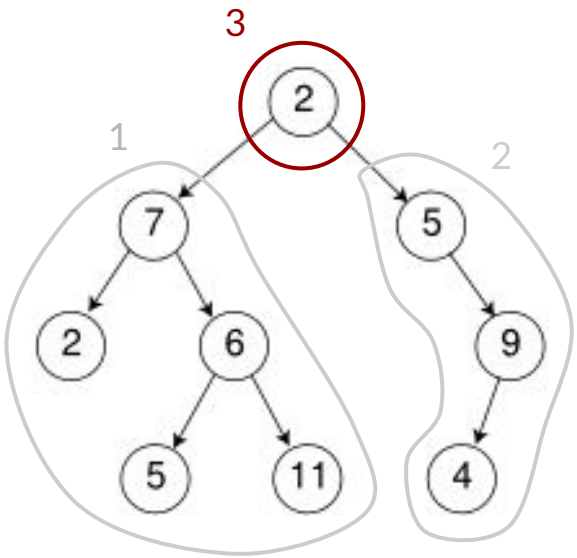
- **Visita** raiz
- Percorre sub-árvore esquerda em pré-ordem
- Percorre sub-árvore direita em pré-ordem

Em-ordem (simétrica) (ESQ, RAIZ, DIR)

- Percorre sub-árvore esquerda em ordem simétrica
- **Visita** raiz
- Percorre sub-árvore direita em ordem simétrica

Pós-ordem (ESQ, DIR, RAIZ)

- Percorre sub-árvore esquerda em pós-ordem
- Percorre sub-árvore direita em pós-ordem
- **Visita** raiz



Formas de Percursos

Pré-ordem (RAIZ, ESQ, DIR)

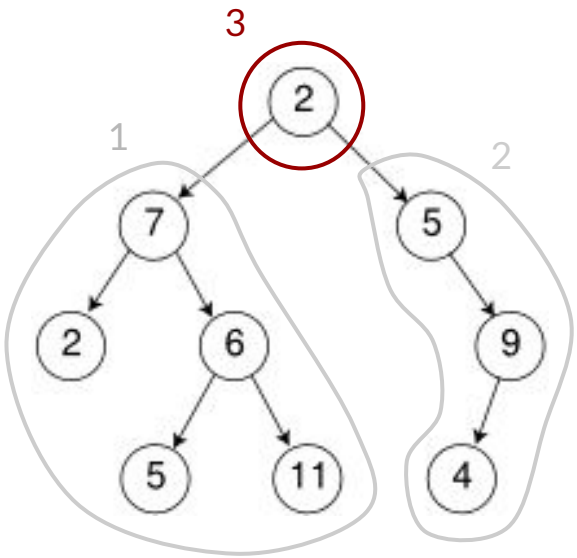
- **Visita** raiz
- Percorre sub-árvore esquerda em pré-ordem
- Percorre sub-árvore direita em pré-ordem

Em-ordem (simétrica) (ESQ, RAIZ, DIR)

- Percorre sub-árvore esquerda em ordem simétrica
- **Visita** raiz
- Percorre sub-árvore direita em ordem simétrica

Pós-ordem (ESQ, DIR, RAIZ)

- Percorre sub-árvore esquerda em pós-ordem
- Percorre sub-árvore direita em pós-ordem
- **Visita** raiz



simulação

Formas de Percursos

Pré-ordem (RAIZ, ESQ, DIR)

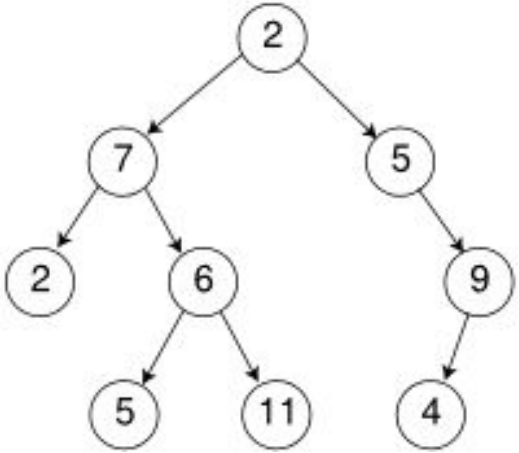
- **Visita** raiz
- Percorre sub-árvore esquerda em pré-ordem
- Percorre sub-árvore direita em pré-ordem

Em-ordem (simétrica) (ESQ, RAIZ, DIR)

- Percorre sub-árvore esquerda em ordem simétrica
- **Visita** raiz
- Percorre sub-árvore direita em ordem simétrica

Pós-ordem (ESQ, DIR, RAIZ)

- Percorre sub-árvore esquerda em pós-ordem
- Percorre sub-árvore direita em pós-ordem
- **Visita** raiz



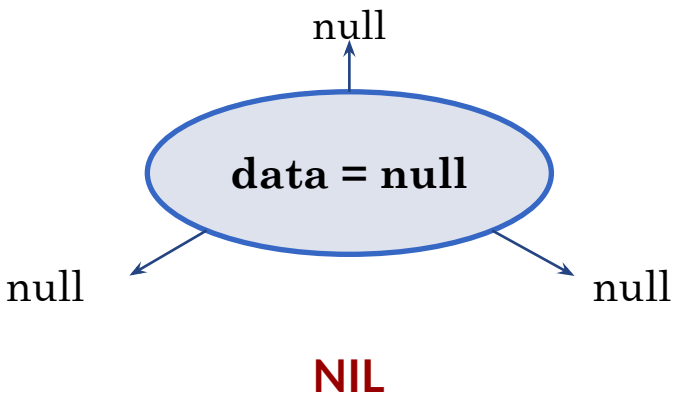
2 5 11 6 7 4 9 5 2

simulação

Árvore Binária
Implementação

Árvore Binária (Binary Tree)

Estrutura básica dos nós

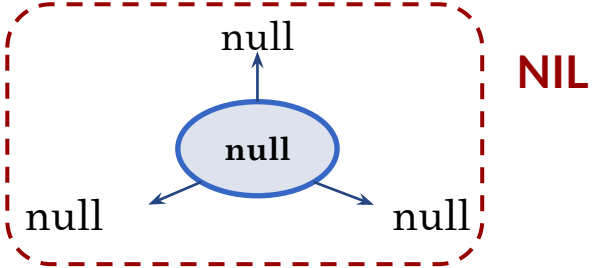


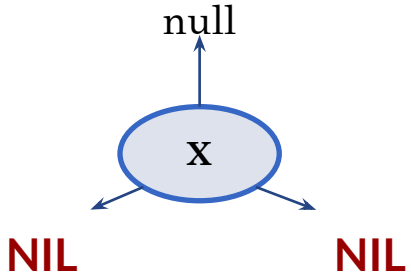

```
public class BTNode<T> {
  protected T data;
  protected BTNode<T> left;
  protected BTNode<T> right;
  protected BTNode<T> parent;

  public boolean isEmpty(){
    return this.data == null;
  }

  //getters, setters, equals, toString
}
```

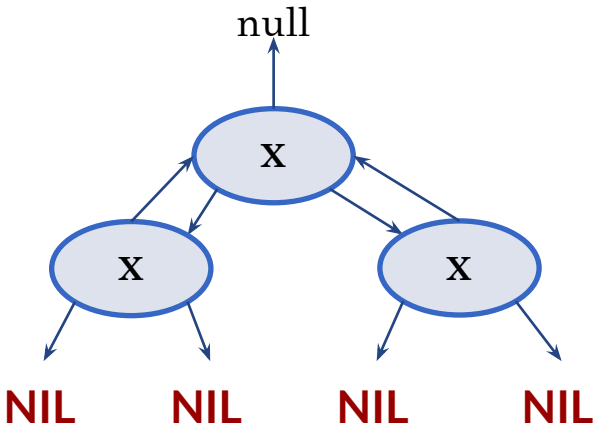
Árvore Binária (Binary Tree) - Serviços

```
public interface BT<T> {
  public BTNode<T> getRoot();
  public boolean isEmpty();
  public int height();
  public BTNode<T> search(T elem);
  public void insert(T value);
  public void remove(T key);
  public T[] preOrder();
  public T[] order();
  public T[] postOrder();
  public int size();
}
```

Árvore Binária (Binary Tree) - Exemplos





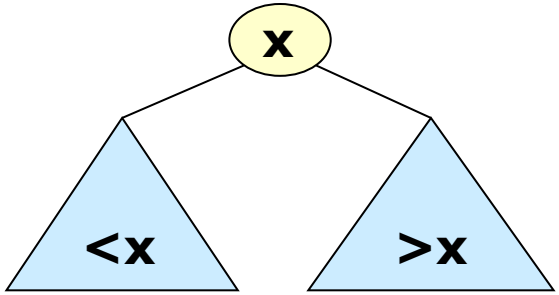


Árvore Binária de Busca
Binary Search Tree

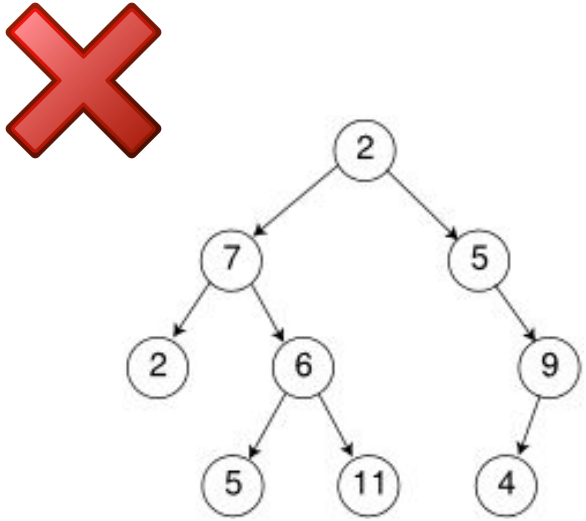
Árvore Binária de Busca (BST)

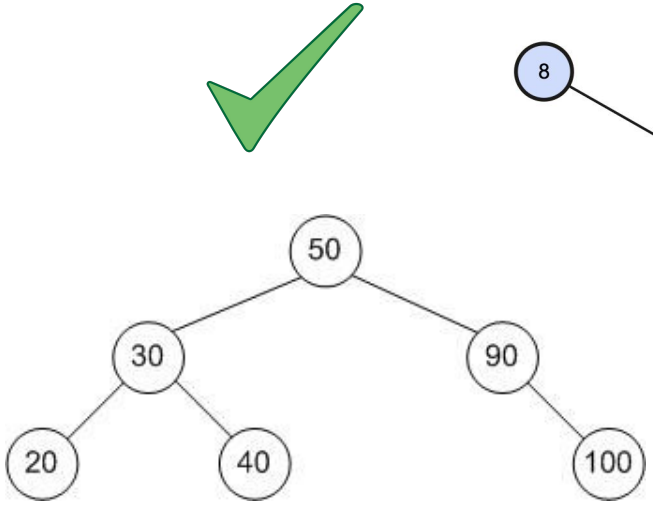
Árvore binária com noção de ordem (dados comparáveis)

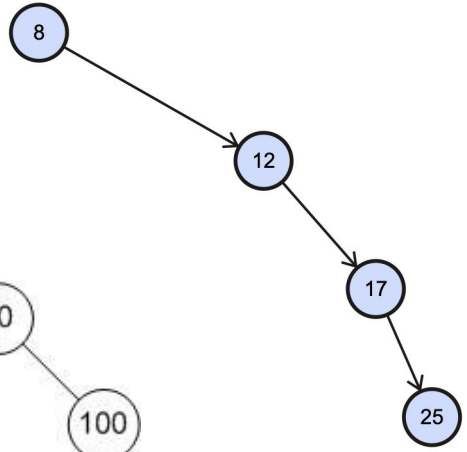
Invariante/propriedade:



Árvore Binária de Busca (BST) - Exemplos







Árvore Binária de Pesquisa (BST) - Implementação

Nó de BT x Nó de BST

Árvore Binária de Pesquisa (BST) - Implementação

Nó de BT x Nó de BST

BST guarda tipos comparáveis/ordenáveis

Árvore Binária de Pesquisa (BST) - Implementação

Nó de BT x Nó de BST

BST guarda tipos comparáveis/ordenáveis

```
public class BSTNode<T extends Comparable<T>>
  extends BTNode<T> {




}
```

Árvore Binária de Pesquisa (BST) - Implementação

Nó de BT x Nó de BST

Uma BST é uma BT?

Árvore Binária de Pesquisa (BST) - Implementação

Nó de BT x Nó de BST

Uma BST é uma BT? SIM, com mais serviços!

Árvore Binária de Pesquisa (BST) - Implementação

Nó de BT x Nó de BST

Uma BST é uma BT? SIM, com mais serviços!

```
public interface BST<T extends Comparable<T>> extends BT<T> {

  public BTNode<T> maximum();
  public BTNode<T> minimum();
  public BTNode<T> successor(BTNode<T>  node);
  public BTNode<T> predecessor(BTNode<T>  node);
}
```

Árvore Binária de Busca (BST) Implementação

Usar sempre que possível o template da recursão!!!

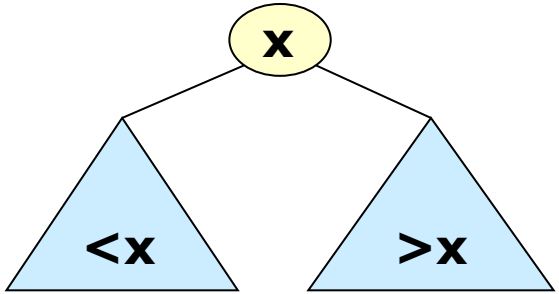
Início **sempre** pela raiz

Busca/Pesquisa

Caso base 1: árvore vazia → dado não pertence

Caso base 2: nó guarda o dado → pertence

Caso indutivo: pesquisar no filho à esquerda ou no filho à direita



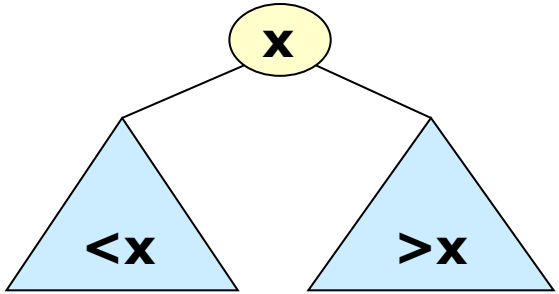
Busca/Pesquisa

Caso base 1: árvore vazia → dado não pertence

Caso base 2: nó guarda o dado → pertence

Caso indutivo: pesquisar no filho à esquerda ou no filho à direita

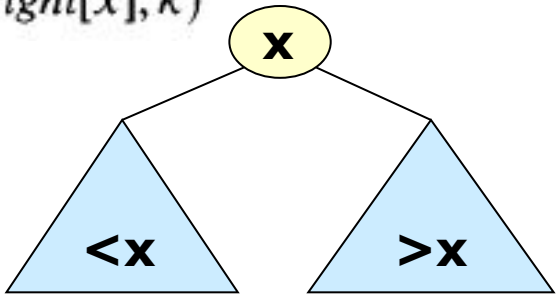
Qual a redução do espaço de busca a cada chamada recursiva?



Busca/Pesquisa

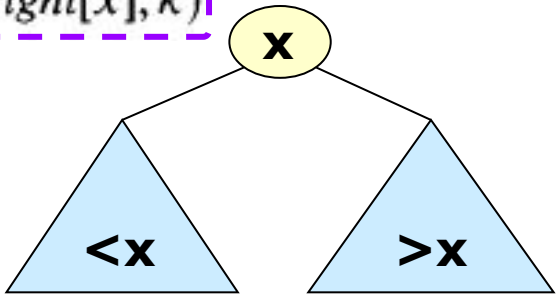
Qual o custo?

```
TREE-SEARCH(x, k)
if x = NIL or k = key[x]
    then return x
if k < key[x]
    then return TREE-SEARCH(left[x], k)
    else return TREE-SEARCH(right[x], k)
```



Busca/Pesquisa

```
TREE-SEARCH(x, k)
if x = NIL or k = key[x]
    then return x
if k < key[x]
    then return TREE-SEARCH(left[x], k)
    else return TREE-SEARCH(right[x], k)
```



Qual o custo?

$T(n) = T(n/2) + O(1)$

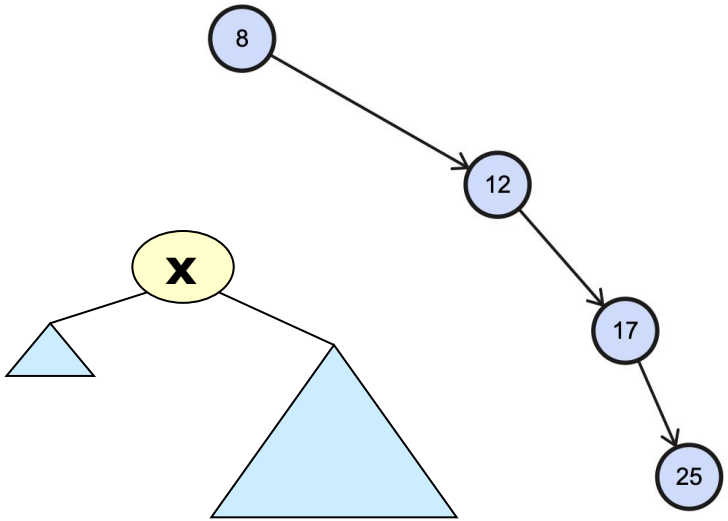
$\Downarrow$

$\Theta(\log n)$

Busca/Pesquisa

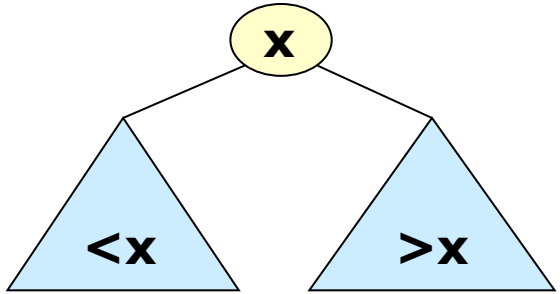
```
TREE-SEARCH(x, k)
if x = NIL or k = key[x]
    then return x
if k < key[x]
    then return TREE-SEARCH(left[x], k)
    else return TREE-SEARCH(right[x], k)
```

O que acontece com a busca?



Minimum

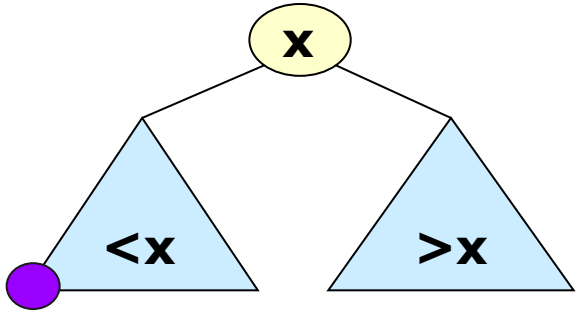
Onde está o elemento mínimo de uma BST?



Minimum

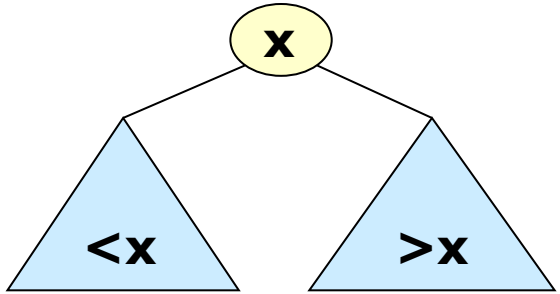
Onde está o elemento mínimo de uma BST?

```
TREE-MINIMUM(x)
while left[x] ≠ NIL
      do x ← left[x]
return x
```



Maximum

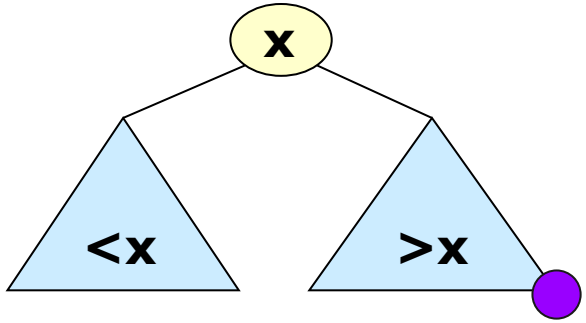
Onde está o elemento máximo de uma BST?



Maximum

Onde está o elemento máximo de uma BST?

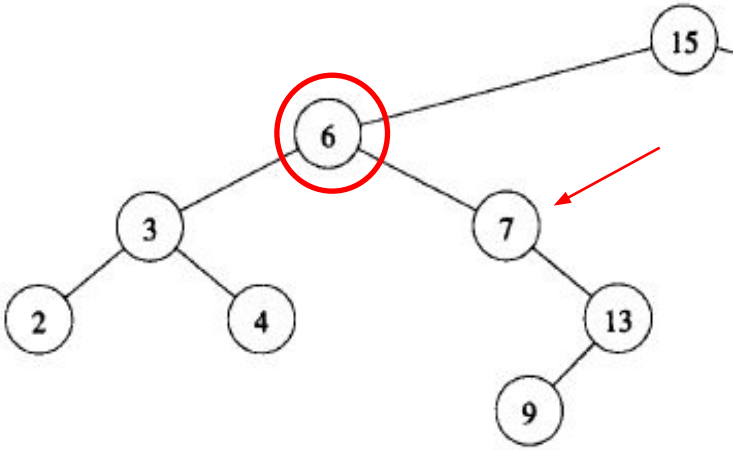
```
TREE-MAXIMUM(x)
while right[x] ≠ NIL
      do x ← right[x]
return x
```



Sucessor

Como buscar o sucessor de um elemento em uma BST?

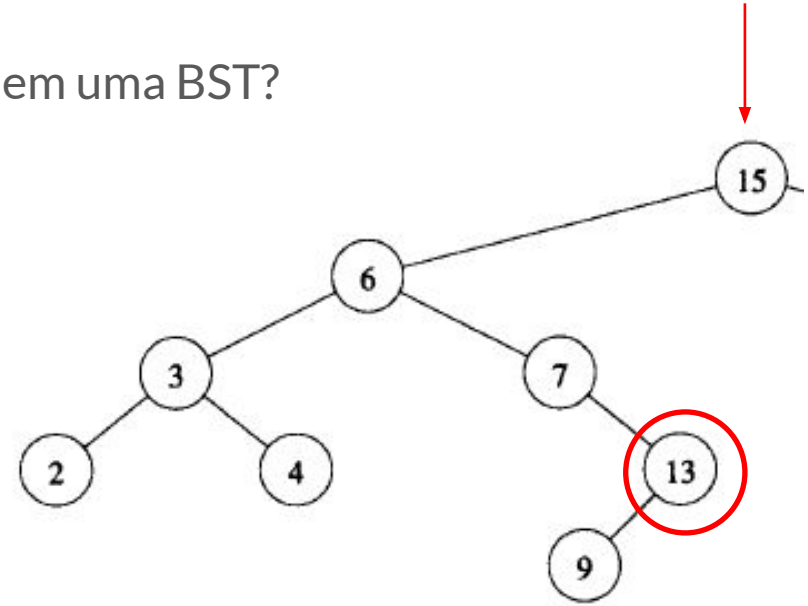
```
TREE-SUCCESSOR(x)
if right[x] ≠ NIL
    then return TREE-MINIMUM(right[x])
y ← p[x]
while y ≠ NIL and x = right[y]
        do x ← y
           y ← p[y]
return y
```



Sucessor

Como buscar o sucessor de um elemento em uma BST?

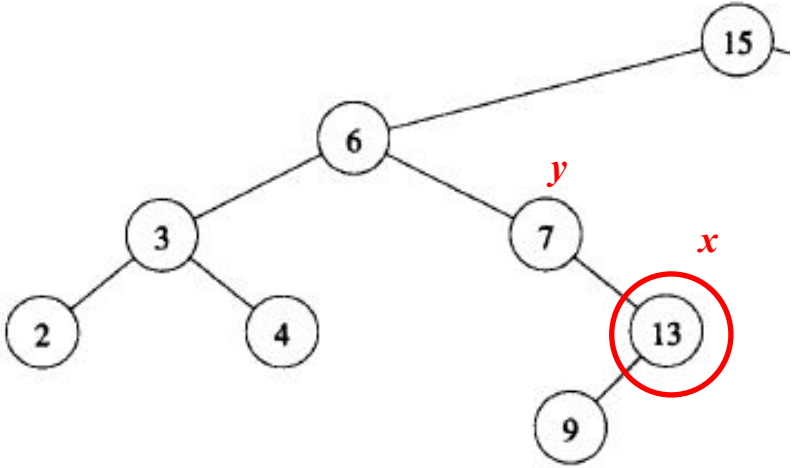
```
TREE-SUCCESSOR(x)
if right[x] ≠ NIL
    then return TREE-MINIMUM(right[x])
y ← p[x]
while y ≠ NIL and x = right[y]
        do x ← y
           y ← p[y]
return y
```



Sucessor

Como buscar o sucessor de um elemento em uma BST?

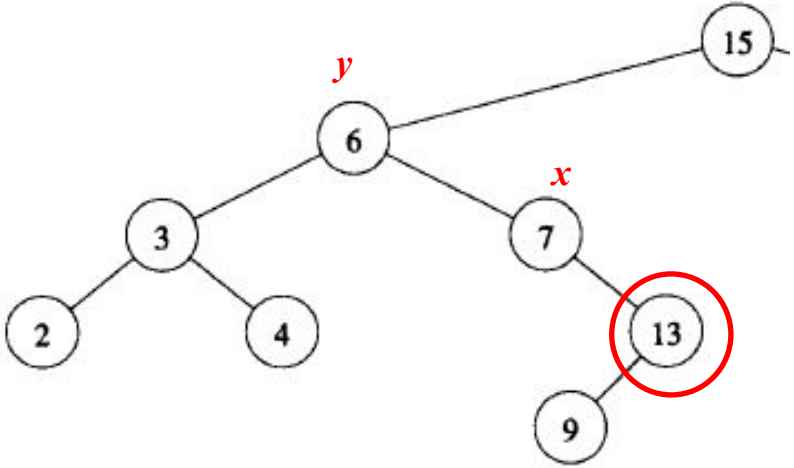
```
TREE-SUCCESSOR(x)
if right[x] ≠ NIL
    then return TREE-MINIMUM(right[x])
y ← p[x]
while y ≠ NIL and x = right[y]
        do x ← y
           y ← p[y]
return y
```



Sucessor

Como buscar o sucessor de um elemento em uma BST?

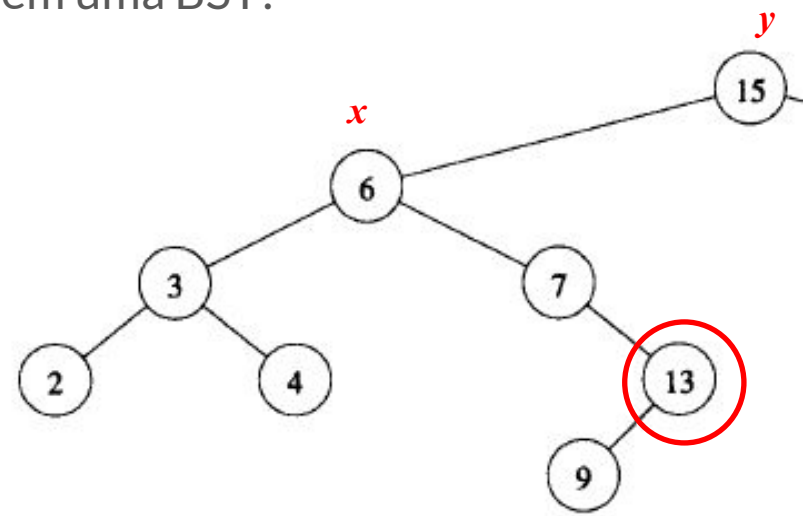
```
TREE-SUCCESSOR(x)
if right[x] ≠ NIL
    then return TREE-MINIMUM(right[x])
y ← p[x]
while y ≠ NIL and x = right[y]
      do x ← y
         y ← p[y]
return y
```



Sucessor

Como buscar o sucessor de um elemento em uma BST?

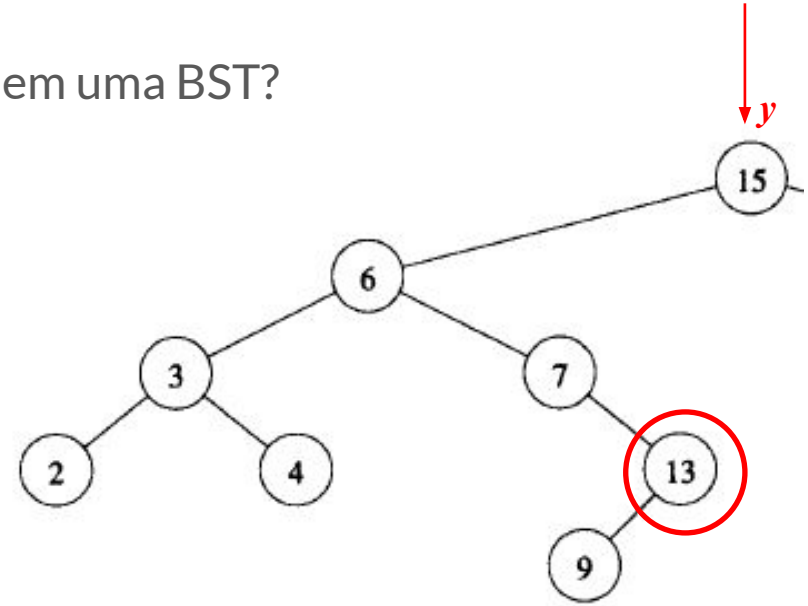
```
TREE-SUCCESSOR(x)
if right[x] ≠ NIL
    then return TREE-MINIMUM(right[x])
y ← p[x]
while y ≠ NIL and x = right[y]
        do x ← y
           y ← p[y]
return y
```



Sucessor

Como buscar o sucessor de um elemento em uma BST?

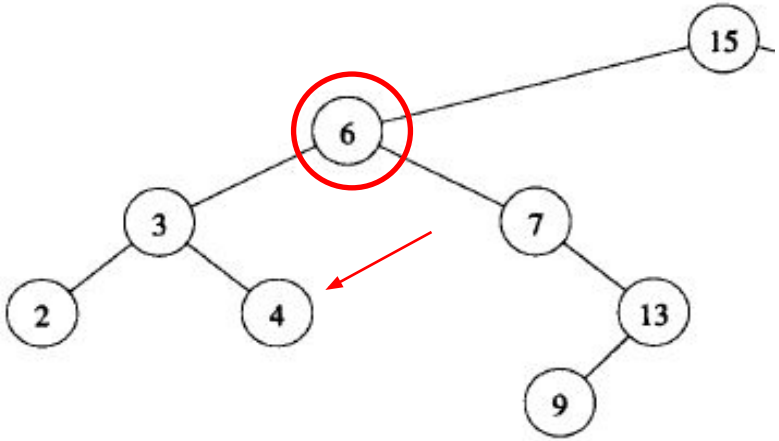
```
TREE-SUCCESSOR(x)
if right[x] ≠ NIL
    then return TREE-MINIMUM(right[x])
y ← p[x]
while y ≠ NIL and x = right[y]
        do x ← y
           y ← p[y]
return y
```



Predecessor

Como buscar o predecessor de um elemento em uma BST?

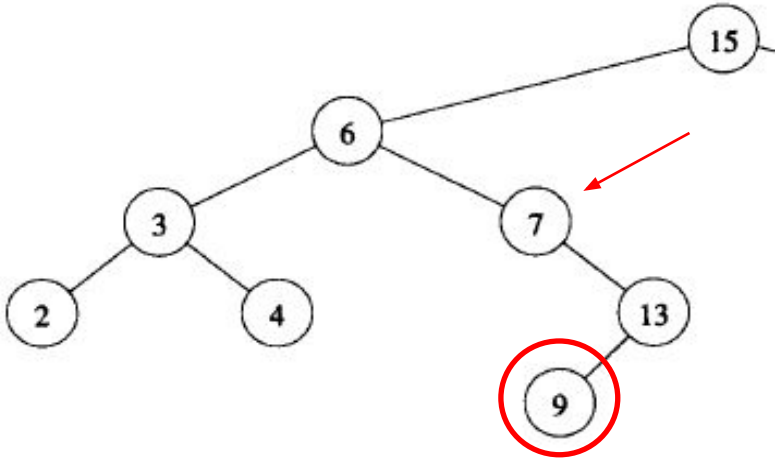
```
Tree-Predecessor(x)
 if left[x] != NIL
    then return Tree-Maximum(left[x])
 y = p[x]
 while y != NIL and x = left[y]
   do x = y
        y = p[y]
 return y
```



Predecessor

Como buscar o predecessor de um elemento em uma BST?

```
Tree-Predecessor(x)
 if left[x] != NIL
    then return Tree-Maximum(left[x])
 y = p[x]
 while y != NIL and x = left[y]
   do x = y
       y = p[y]
 return y
```



Inserção

Caso base: nó vazio → insere o novo dado (sempre em um NIL)

Caso indutivo: nó não-vazio → insere no filho à esquerda ou no filho à direita

Não se preocupar com duplicatas

Simulação

Inserção

```
Tree-insert(BSTNode node, T element){
  if(node=NIL){
    node.data = element
    node.left = NIL
    node.right = NIL
  }else{
      if(element< node.data){
        Tree-insert(node.left, element)
      }else if (element > node.data){
        Tree-insert (node.right, element)
      }
  }
}
```

Inserção

```
Tree-insert(BSTNode node, T element){
   if(node=NIL){
      node.data = element
      node.left = NIL
      node.right = NIL
   }else{
        if(element< node.data){
           Tree-insert(node.left, element)
        }else if (element > node.data){
           Tree-insert (node.right, element)
        }
   }
}
```

$T(n) = T(n/2) + O(1)$

$\Downarrow$

$\Theta(\log n)$

Remoção

Caso base 1: nó vazio → nada a fazer

Caso base 2: nó a ser removido é folha → remover folha

Caso base 3: nó a ser removido tem um filho → conecta o filho com o pai do nó

Caso indutivo: traz o sucessor para o local do nó a ser removido e remove recursivamente o sucessor.

Simulação

Remoção

```
Tree-remove(value) {
  BSTNode node = search(value)
  if(node != NIL){
    if(node is leaf){ node = NIL}
    else if (node has one child){
      if node != root
        if(node is left child){
            if(node.left != NIL)
                node.left is left child of node.parent
            else
                node.right is left child of node.parent
        else  //node is right child
            if(node.left != NIL)
                node.left is right child of parent
            else
                node.right is right child of parent
      else
        root = not NIL child of root
    }else{
        BSTNode sucessor = sucessor(node);
        node.data = sucessor.data;
        Tree-remove(sucessor.data);
    }
```

Percursos

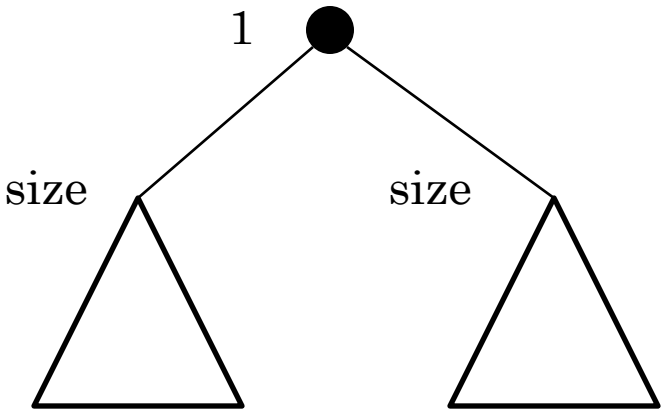
```
preOrder(BSTNode node){
  if(node != NIL){
    visit(node);
    preOrder(node.left);
    preOrder(node.right);
  }
}

visit(BSTNode){
  print(node.key);
}
```

```
order(BSTNode node){
  if(node != NIL){
    order(node.left);
    visit(node);
    order(node.right);
  }
}

visit(BSTNode){
  print(node.key);
}
```

```
postOrder(BSTNode node){
  if(node != NIL){
    postOrder(node.left);
    postOrder(node.right);
    visit(node);
  }
}

visit(BSTNode){
  print(node.key);
}
```

Tamanho (size)

size(root) = 1 + size(left) + size(right)




```
int size(){
  return size(root)
}

int size(BSTNode node){
  if(node.isEmpty)
      return 0
  else
      return 1 + size(node.left) +  size(node.right)
}
```

Problemas pertinentes de BST

- Como calcular o floor de um número, dado um array de elementos, usando uma BST?
- Como calcular a k-ésima estatística de ordem usando uma BST?
- Como preencher uma BST com elementos de um array ordenado de forma a deixar a BST balanceada?